

U.T. 2. Manejo de la sintaxis del lenguaje.

La sintaxis de un lenguaje de programación se define como el conjunto de reglas que deben seguirse al escribir el código fuente de los programas para considerarse como correctos para ese lenguaje de programación.

2.1. Características de JavaScript

- Sintaxis parecida a la C++ y la de Java.
- Basado en el concepto de objeto, pero no orientado a objetos
- No se define el tipo de las variables. (Débilmente tipado)
- Las variables se declaran dinámicamente asignándoles un valor o alternativamente precediéndolas con la palabra var
- Las variables declaradas con var dentro de una función son locales

2.2. Hola Mundo en Javascript

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=iso-
8859-1" />
    <title>Hola Mundo</title>
  </head>
  <body>
    <p>
      <script type="text/javascript">
        document.write(";Hola mundo!");
      </script>
    </p>
  </body>
</html>
```

2.3. El lenguaje JavaScript: sintaxis

La sintaxis de JavaScript es muy similar a la de otros lenguajes de programación como Java y C. Las normas básicas que definen la sintaxis de JavaScript son las siguientes:

2.3.1. Mayúsculas y minúsculas

Es sensible a mayúsculas y minúsculas

2.3.2. Comentarios en el código

Los comentarios siguen la misma sintaxis que en C++, `/**/` y `//`

2.3.3. Tabulación y saltos de línea

Ignora espacios, tabulaciones y saltos de línea. ¡Ojo! Por asuntos de presentación y legibilidad de código es importante utilizarlos.

2.3.4. El punto y coma

El “;” no es necesario al final de cada sentencia si las hacemos en diferentes líneas de código, aunque es conveniente hacerlo por asuntos de legibilidad.

2.3.5. Palabras reservadas

JavaScript utiliza una serie de palabras para crear las instrucciones que forman cada programa. Estas palabras son reservadas y no se pueden utilizar como nombre de una variable o función.

El estándar ECMA-262 incluye la lista de las palabras reservadas que utiliza actualmente JavaScript y la lista de las palabras reservadas para su uso futuro.

Utilizadas actualmente:

```
break, else, new, var, case, finally, return, void, catch, for,  
switch, while, continue, function, this, with, default, if, throw,  
delete, in, try, do, instanceof, typeof
```

Reservadas para su uso futuro:

```
abstract, enum, int, short, boolean, export, interface, static, byte,
extends, long, super, char, final, native, synchronized, class, float,
package, throws, const, goto, private, transient, debugger,
implements, protected, volatile, double, import, public
```

2.4. Tipos de datos

Los tipos de datos nos especifican los tipos de valores que puede tomar una variable, y que operaciones se van a poder realizar con ella.

Los tipos de datos soportados por JavaScript son los siguientes:

2.4.1. Números

Las variables numéricas permiten almacenar cualquier valor numérico. Si el número es entero, se indica directamente. Si el número es decimal, se debe utilizar el punto (.) para separar la parte entera de la decimal.

```
var variable1 = 10;
var variable2 = 3.14159265;
```

Además del sistema numérico decimal, también se pueden indicar valores en el sistema octal (si se incluye un cero delante del número) y en sistema hexadecimal (si se incluye un cero y una x delante del número).

```
var variable1 = 10;
var variable_octal = 034;
var variable_hexadecimal = 0xA3;
```

JavaScript define tres valores especiales muy útiles cuando se trabaja con números. En primer lugar se definen los valores Infinity y -Infinity para representar números demasiado grandes (positivos y negativos) y con los que JavaScript no puede trabajar.

```
var variable1 = 3, variable2 = 0;
alert(variable1/variable2); // muestra "Infinity"
```

El otro valor especial definido por JavaScript es NaN, que es el acrónimo de "Not a Number". De esta forma, si se realizan operaciones matemáticas con variables no numéricas, el resultado será de tipo NaN.

Para manejar los valores NaN, se utiliza la función relacionada `isNaN()`, que devuelve `true` si el parámetro que se le pasa no es un número:

```
var variable1 = 3;
var variable2 = "hola";
isNaN(variable1); // false
isNaN(variable2); // true
isNaN(variable1 + variable2); // true
```

Por último, JavaScript define algunas constantes matemáticas que representan valores numéricos significativos:

Constante	Valor	Significado
Math.E	2.718281828459045	Constante de Euler, base de los logaritmos naturales y también llamado número e
Math.LN2	0.6931471805599453	Logaritmo natural de 2
Math.LN10	2.302585092994046	Logaritmo natural de 10
Math.LOG2E	1.4426950408889634	Logaritmo en base 2 de Math.E
Math.LOG10E	0.4342944819032518	Logaritmo en base 10 de Math.E
Math.PI	3.141592653589793	Pi, relación entre el radio de una circunferencia y su diámetro
Math.SQRT1_2	0.7071067811865476	Raíz cuadrada de 1/2
Math.SQRT2	1.4142135623730951	Raíz cuadrada de 2

De esta forma, para calcular el área de un círculo de radio `r`, se debe utilizar la constante que representa al número Pi:

```
var area = Math.PI * r * r;
```

2.4.2. Cadenas de texto

Las variables de tipo cadena de texto permiten almacenar cualquier sucesión de caracteres. Cada carácter de la cadena se encuentra en una posición a la que se puede acceder individualmente, siendo el primer carácter el de la posición 0.

El valor de las cadenas de texto se indica encerrado entre comillas simples o dobles:

```
var variable1 = "hola";  
var variable2 = 'mundo';  
var variable3 = "hola mundo, esta es una frase más larga";
```

Las cadenas de texto pueden almacenar cualquier carácter, aunque algunos no se pueden incluir directamente en la declaración de la variable. Si por ejemplo se incluye un ENTER para mostrar el resto de caracteres en la línea siguiente, se produce un error en la aplicación:

```
var variable = "hola mundo, esta es  
una frase más larga";
```

La variable anterior no está correctamente definida y se producirá un error en la aplicación. Por tanto, resulta evidente que algunos caracteres *especiales* no se pueden incluir directamente. De la misma forma, como las comillas (doble y simple) se utilizan para encerrar los contenidos, también se pueden producir errores:

```
var variable1 = "hola 'mundo'";  
var variable2 = 'hola "mundo"';  
var variable3 = "hola 'mundo', esta es una \"frase\" más larga";
```

Si el contenido de texto tiene en su interior alguna comilla simple, se encierran los contenidos con comillas dobles (como en el caso de la variable1 anterior). Si el contenido de texto tiene en su interior alguna comilla doble, se encierran sus contenidos con comillas simples (como en el caso de la variable2 anterior). Sin embargo, en el caso de la variable3 su contenido tiene tanto comillas simples como comillas dobles, por lo que su declaración provocará un error. Para resolver estos problemas, JavaScript define un mecanismo para incluir de forma sencilla caracteres especiales (ENTER, Tabulador) y problemáticos (comillas). Esta estrategia se denomina "*mecanismo de escape*", ya que se sustituyen los caracteres problemáticos por otros caracteres seguros que siempre empiezan con la barra \:

Si se quiere incluir...	Se debe sustituir por...
Una nueva línea	\n
Un tabulador	\t
Una comilla simple	\'
Una comilla doble	\"
Una barra inclinada	\\

Utilizando el mecanismo de escape, se pueden corregir los ejemplos anteriores:

```
var variable = "hola mundo, esta es \n una frase más larga";
var variable3 = "hola 'mundo', esta es una \"frase\" más larga";
```

2.4.3. Valores booleanos

Además de variables de tipo *boolean*, también se suelen llamar variables lógicas y variables *booleanas*. Se trata de una variable que sólo puede almacenar uno de los dos valores especiales definidos y que representan el valor "*verdadero*" y el valor "*falso*".

```
var variable1 = true;
var variable2 = false;
```

Los valores true y false son valores especiales, de forma que no son palabras ni números ni ningún otro tipo de valor.

Cuando es necesario convertir una variable numérica a una variable de tipo boolean, JavaScript aplica la siguiente conversión: el número 0 se convierte en false y cualquier otro número distinto de 0 se convierte en true.

2.5. Variables

2.5.1. Declaración de variables

Las variables se definen mediante la palabra reservada `var`, que permite definir una o varias variables simultáneamente:

```
var variable1;  
var variable2, variable3;
```

El nombre de las variables debe cumplir las dos siguientes condiciones:

- El primer carácter debe ser una letra o un guión bajo (`_`) o un dólar (`$`).
- El resto de caracteres pueden ser letras, números, guiones bajos (`_`) y símbolos de dólar (`$`).

2.5.2. Inicialización de variables

No es obligatorio inicializar una variable al declararla:

```
var variable6;
```

Si la variable no se declara mediante el operador `var`, automáticamente se crea una variable global con ese identificador y su valor. Ejemplo:

```
var variable1 = 16;  
variable2 = variable1 + 4;
```

En el ejemplo anterior, la `variable2` no ha sido declarada, por lo que al llegar a esa instrucción, JavaScript crea automáticamente una variable global llamada `variable2` y le asigna el valor correspondiente.

2.5.3. Ámbito de variables

El ámbito de una variable (llamado *scope* en inglés) es la zona del programa en la que se define la variable. JavaScript define dos ámbitos para las variables: global y local.

Variable local

```
function muestraMensaje() {  
    var mensaje = "Mensaje de prueba";  
}  
muestraMensaje();  
alert(mensaje);
```

Cuando se ejecuta el código JavaScript anterior no se muestra por pantalla ningún mensaje. La variable mensaje se ha definido dentro de la función y por tanto es una variable local que solamente está definida dentro de la función.

Cualquier instrucción que se encuentre dentro de la función puede hacer uso de la variable. Sin embargo, cualquier instrucción que se encuentre en otras funciones o fuera de cualquier función no tendrá definida la variable mensaje.

Variable global

Variable global es la que está definida en cualquier punto del programa (incluso dentro de cualquier función).

```
var mensaje = "Mensaje de prueba";  
function muestraMensaje() {  
    alert(mensaje);  
}
```

El código JavaScript anterior define una variable fuera de cualquier función. Este tipo de variables automáticamente se transforman en variables globales y están disponibles en cualquier punto del programa.

Si una variable se declara fuera de cualquier función, automáticamente se transforma en variable global independientemente de si se define utilizando la palabra reservada var o no. Sin embargo, en el interior de una función, las variables declaradas mediante var se consideran locales y el resto se transforman también automáticamente en variables globales.

Por lo tanto, el siguiente ejemplo si que funciona como se espera:

```
function muestraMensaje() {  
    mensaje = "Mensaje de prueba";  
}  
muestraMensaje();  
alert(mensaje);
```

En caso de colisión entre las variables globales y locales, dentro de una función prevalecen las variables locales:

```
var mensaje = "gana la de fuera";  
function muestraMensaje() {  
    var mensaje = "gana la de dentro";  
    alert(mensaje);  
}  
alert(mensaje);  
muestraMensaje();  
alert(mensaje);
```

El código anterior muestra por pantalla los siguientes mensajes:

gana la de fuera
gana la de dentro
gana la de fuera

La variable local llamada mensaje dentro de la función tiene más prioridad que la variable global del mismo nombre, pero solamente dentro de la función.

Si no se define la variable dentro de la función con la palabra reservada var, en realidad se está modificando el valor de la variable global:

```
var mensaje = "gana la de fuera";  
function muestraMensaje() {  
    mensaje = "gana la de dentro";  
    alert(mensaje);  
}
```

```
alert(mensaje);  
muestraMensaje();  
alert(mensaje);
```

En este caso, los mensajes mostrados son:

gana la de fuera
gana la de dentro
gana la de dentro

La recomendación general es definir como variables locales todas las variables que sean de uso exclusivo para realizar las tareas encargadas a cada función. Las variables globales se utilizan para compartir variables entre funciones de forma rápida.

2.1. Operadores

Los operadores son los elementos básicos que se utilizan para modificar el valor de las variables y para combinar varios valores entre sí para obtener otro valor.

JavaScript define numerosos operadores que se detallan a continuación.

2.1.1. Operadores aritméticos

JavaScript permite realizar manipulaciones matemáticas sobre el valor de las variables numéricas. Los operadores definidos son: **suma (+)**, **resta (-)**, **multiplicación (*)** y **división (/)**.

```
var numero1 = 10;  
var numero2 = 5;  
resultado = numero1 / numero2; // resultado = 2  
resultado = 3 + numero1;       // resultado = 13  
resultado = numero2 - 4;       // resultado = 1  
resultado = numero1 * numero 2; // resultado = 50
```

Otro operador aritméticos es el **módulo**, que calcula el resto de la división entera. Si se divide 10 entre 5, la división es exacta y da un resultado de 2. El resto de esa división es 0, por lo que *"módulo de 10 y 5"* es igual a 0.

Sin embargo, si se divide 9 y 5, la división no es exacta, el resultado es 1 y el resto es 4, por lo que *"módulo de 9 y 5"* es igual a 4.

El módulo en JavaScript se indica mediante el símbolo %, que no debe confundirse con el porcentaje:

```
var numero1 = 10;
var numero2 = 5;
resultado = numero1 % numero2; // resultado = 0
numero1 = 9;
numero2 = 5;
resultado = numero1 % numero2; // resultado = 4
```

Los operadores aritméticos se pueden **combinar** con el operador de asignación para escribir de forma abreviada algunas operaciones comunes:

```
var numero1 = 5;
numero1 += 3; // numero1 = numero1 + 3 = 8
numero1 -= 1; // numero1 = numero1 - 1 = 4
numero1 *=2;  // numero1 = numero1 * 2 = 10
numero1 /= 2; // numero1 = numero1 / 2 = 2.5
numero1 %= 3; // numero1 = numero1 % 3 = 2
```

Otros dos operadores aritméticos de JavaScript son el **incremento** y el **decremento**. Solamente son válidos para las variables numéricas y son un método sencillo de incrementar o decrementar en 1 unidad el valor de una variable, tal y como se muestra en el siguiente ejemplo:

```
var numero = 5;
++numero;
alert(numero); // numero = 6
```

El anterior ejemplo es equivalente a:

```
var numero = 5;
numero = numero + 1;
alert(numero); // numero = 6
```

De la misma forma, el operador -- se utiliza para decrementar el valor de la variable:

```
var numero = 5;
--numero;
alert(numero); // numero = 4
```

Como ya se supone, el anterior ejemplo es equivalente a:

```
var numero = 5;
numero = numero - 1;
alert(numero); // numero = 4
```

Además de estos dos operadores, existen otros dos operadores similares pero que se diferencian en la forma en la que se realiza el incremento o decremento. En el siguiente ejemplo:

```
var numero = 5;
numero++;
alert(numero); // numero = 6
```

El resultado es el mismo que antes y puede parecer que es equivalente añadir el operador ++ delante o detrás del identificador de la variable. Sin embargo, el siguiente ejemplo muestra sus diferencias:

```
var numero1 = 5;
var numero2 = 2;
numero3 = numero1++ + numero2;
// numero3 = 7, numero1 = 6
var numero1 = 5;
var numero2 = 2;
```

```
numero3 = ++numero1 + numero2;  
// numero3 = 8, numero1 = 6
```

Si el operador ++ se indica como prefijo del identificador de la variable, su valor se incrementa antes de realizar cualquier otra operación. Si el operador ++ se indica como sufijo del identificador de la variable, su valor se incrementa después de ejecutar la sentencia en la que aparece.

2.1.2. Operadores lógicos

Uno de los operadores lógicos más utilizados es el de la **negación**. Se utiliza para obtener el valor lógico contrario al valor de la variable:

```
var visible = true;  
alert(!visible); // Muestra 'false' y no 'true'
```

La negación lógica se obtiene prefijando el símbolo ! al identificador de la variable. Cuando la variable es de tipo booleano, obtener su valor lógico contrario es trivial:

variable	!variable
true	false
false	true

Por el contrario, si la variable almacena un número o una cadena de texto, no se puede obtener su valor lógico contrario de forma directa. En este caso, JavaScript convierte previamente la variable a un valor lógico y después obtiene su valor contrario.

Si la variable original contiene un número, su transformación en variable lógica es false si el número es 0 y true en cualquier otro caso. Si la variable original contiene una cadena de texto, su transformación en variable lógica es false si la cadena no contiene ningún carácter y true en cualquier otro caso:

```

var cantidad = 0;
vacio = !cantidad; // vacio = true
cantidad = 2;
vacio = !cantidad; // vacio = false
var mensaje = "";
sinMensaje = !mensaje; // sinMensaje = true
mensaje = "hola mundo";
sinMensaje = !mensaje; // sinMensaje = false

```

La operación lógica **AND** combina dos valores booleanos para obtener como resultado otro valor de tipo lógico. El resultado de la operación solamente es true si los dos operandos son true. El operador se indica mediante el símbolo **&&**:

variable1	variable2	variable1 && variable2
true	true	true
true	false	false
false	true	false
false	false	false

El siguiente ejemplo muestra cómo combinar valores mediante el operador &&:

```

var valor1 = true;
var valor2 = false;
resultado = valor1 && valor2; // resultado = false
valor1 = true;
valor2 = true;
resultado = valor1 && valor2; // resultado = true

```

La operación lógica **OR** también combina dos valores booleanos para obtener como resultado otro valor de tipo lógico. El resultado de la operación es true si alguno de los dos operandos es true. El operador se indica mediante el símbolo **||**:

variable1	variable2	variable1 variable2
true	true	true

true	false	true
false	true	true
false	false	false

El siguiente ejemplo muestra cómo combinar valores mediante el operador ||:

```
var valor1 = true;
var valor2 = false;
resultado = valor1 || valor2; // resultado = true
valor1 = false;
valor2 = false;
resultado = valor1 || valor2; // resultado = false
```

2.1.3. Operadores de asignación

El operador de asignación es se utiliza para asignar a una variable un valor específico. El símbolo utilizado es = (no confundir con el operador ==)

```
var numero1 = 3;
var variable1 = "hola mundo";
```

2.1.4. Operadores relacionales

Los operadores relacionales son: mayor que (>), menor que (<), mayor o igual (>=), menor o igual (<=), igual (==) y distinto (!=).

El resultado de todas estas operaciones siempre es un valor booleano:

```
var numero1 = 3;
var numero2 = 5;
resultado = numero1 > numero2; // resultado = false
resultado = numero1 < numero2; // resultado = true
numero1 = 5;
numero2 = 5;
resultado = numero1 >= numero2; // resultado = true
resultado = numero1 <= numero2; // resultado = true
```

```
resultado = numero1 == numero2; // resultado = true
resultado = numero1 != numero2; // resultado = false
```

El operador `==` es la mayor fuente de errores de programación incluso para los usuarios que ya tienen cierta experiencia desarrollando scripts. Si se quiere comparar el valor de dos variables, no se debe utilizar el operador `=`:

```
var numero1 = 5;
resultado = numero1 = 3; // numero1 = 3 y resultado = 3
```

Para comparar valores, se debe utilizar el operador `==`

```
var numero1 = 5;
resultado = numero1 == 3; // numero1 = 5 y resultado = false
```

Además de las variables numéricas, también se pueden utilizar variables de tipo cadena de texto con los operadores relacionales:

```
var texto1 = "hola";
var texto2 = "hola";
var texto3 = "adios";
resultado = texto1 == texto3; // resultado = false
resultado = texto1 != texto2; // resultado = false
resultado = texto3 >= texto2; // resultado = false
```

Cuando se comparan cadenas de texto con los operadores `>` y `<`, el resultado obtenido puede ser poco intuitivo. JavaScript compara letra a letra comenzando desde la izquierda hasta que se encuentre una diferencia entre las dos letras. Para determinar si una letra es mayor o menor que otra, se considera que:

$$A < B < \dots < Z < a < b < \dots < z$$

Además de los operadores básicos de igualdad, también existen los operadores *"idéntico"* y *"no idéntico"*. Cuando se utilizan estos operadores,

no sólo se comparan los valores que almacenan las variables, sino que también se compara el tipo de cada variable.

```
var variable1 = 10;
var variable2 = "10";
variable1 == variable2;    // devuelve true
variable1 === variable2;  // devuelve false
```

El operador *"idéntico"* se indica mediante tres signos de igualdad y devuelve true solamente si los dos operandos son exactamente iguales sin necesidad de realizar ninguna conversión.

Si se utiliza el operador ==, JavaScript convierte automáticamente el tipo de las variables para realizar la comparación.

Sin embargo, en el caso del operador *"idéntico"*, las dos variables tienen que ser además del mismo tipo.

El operador *"no idéntico"* tiene un funcionamiento equivalente y se indica mediante los caracteres !==

2.1.5. Operadores condicionales

El operador condicional ?: ejecuta una de dos instrucciones dependiendo de una condición. Es una forma abreviada de una instrucción **if...else**.

evaluación ? sentencia1 : sentencia2

Si evaluación es true ejecuta la sentencia1, si es false ejecuta la sentencia2. Tanto sentencia1 como sentencia2 podrían ser instrucciones compuestas.

```
var numero = prompt("Introduce un numero: ");
(numero < 10) ? alert ("El numero introducido es menor que 10") :
```

```
alert ("El numero introducido es igual o mayor a 10");
```

Equivaldría a:

```
var numero = prompt("Introduce un numero: ");  
if (numero < 10)  
    alert ("El numero introducido es menor que 10")  
else  
    alert ("El numero introducido es igual o mayor a 10");
```

2.2. Estructuras de control de flujo

Las estructuras de control de flujo se dividen en:

1. **Estructuras de decisión condicional:** Las estructuras de decisión condicional nos permiten decidir mediante expresiones condicionales qué sentencias de nuestro código han de ser ejecutadas. Los bucles nos permiten repetir un número determinado o indeterminado de veces una serie de sentencias de nuestro código. (if, switch)
2. **Bucles:** Como ya hemos dicho los bucles nos permiten repetir un número determinado o indeterminado de veces ciertas sentencias de nuestro código (while, for)

2.2.1. Sentencia if

La estructura de decisión condicional principal es **if**:

```
if(expresión condicional)  
{  
    código1  
}  
[else  
{  
    código2  
}]
```

En este caso se evalúa la expresión condicional. Si es *true* se ejecuta el *código1*. Si es falsa se ejecuta el *código2*. El código situado entre corchetes es opcional.

```
nombre = prompt("Introduzca su nombre, por favor", "");
if(nombre == "Alicia")
{
    alert("Hola " + nombre);
}
else
{
    alert("¿Has visto a Alicia?");
}
```

Cuando el código llega a la sentencia **if** comprueba si el valor introducido es realmente 'Alicia' o no. Si es 'Alicia' recibiremos los mensajes pertinentes. En otro caso se nos preguntará por Alicia.

2.2.2. Sentencia switch

Cuando se trata de anidar varias sentencias **if**, JS cuenta con una estructura más especializada:

```
switch(expresión)
{
case valor1:
    código1
    break
case valor2:
    código2
    break
.
.
.
default:
    código
```

```
}
```

En esta estructura se ejecuta el código correspondiente al caso cuya comparación de identidad entre su *valor* y la expresión arroje como resultado *true*. Veamos un ejemplo:

```
nombre = prompt("Introduzca su nombre por favor", "")
switch(nombre)
{
  case "Alicia":
    alert("Hola Alicia")
    break
  case "Lewis":
    alert("Hola Lewis")
    break
  default:
    alert("Hola " + nombre)
}
```

2.2.3. Bucle while

Esta estructura tiene la siguiente sintaxis:

```
while(condicion)
{
  código
}
```

Primero se evalúa que la condición sea verdadera y luego se ejecuta, hasta que la condición pase a ser falsa; una sentencia **while** puede que no se ejecute ni siquiera una vez, si su condición es inicialmente falsa.

```
i = 0
while(i < 5)
{
  alert(i)
  i++
}
```

```
}
```

También se acepta la siguiente variante de esta estructura de control:

```
do
{
    código
}
while(condicion)
```

En esta variante siempre ejecutará el cuerpo del bucle al menos una vez.

En cualquier momento se puede salir de un bucle de JS mediante la palabra clave **break**. Además podemos pasar a la siguiente vuelta de un bucle mediante la palabra clave **continue**. No es aconsejable usarlo por asuntos de legibilidad de código.

2.2.4. Bucle for

Esta estructura tiene la siguiente sintaxis:

```
for(i = inicio; expresión condicional; i++)
{
    código
}
```

En un bucle **for** primeramente se inicializa una variable que recibe el nombre de *contador* representada comúnmente con la letra *i*. A continuación se comprueba la *expresión condicional*. Si es *true* se ejecuta el código correspondiente, se añade una unidad a la variable contador y se vuelve a evaluar la *expresión condicional* y a ejecutar el código, hasta que la *expresión condicional* devuelva *false*.

```
var m
m = ""
for(i = 0; i < 5; i++)
```

```
{  
    m += "vuelta " + i + "\n"  
}  
alert(m)  
alert(i)
```